

- Description

This module manages a wind turbine power monitoring system, ensuring that:

1. **The power output of a wind turbine is continuously measured and compared against expected performance levels.**
2. **If the power output drops below the minimum efficiency threshold, a low-performance alert is triggered.**
3. **If the power output exceeds the maximum safety threshold, an emergency shutdown is recommended.**
4. **A log of all power output readings, alerts, and shutdown events is maintained for operational tracking.**

This system helps optimize wind turbine efficiency, prevent malfunctions, and ensure safe operation.

- Specification

module WindTurbineMonitoringEx

var currentPower, maxPower, minPower: real

var powerLog: sequence of string

process updatePower(changeAmount: real)

functional scenario form:

currentPower >= minPower && powerLog = powerLog $\oplus$ ("Power Updated: " + currentPower + " kW"), Power output updated successfully.

||

currentPower > maxPower && powerLog = powerLog $\oplus$ ("Overload Warning: " + currentPower + " kW, Emergency Shutdown Recommended."), Overload warning triggered, shutdown recommended.

||

currentPower < minPower && powerLog = powerLog $\oplus$ ("Low Efficiency Alert: " + currentPower + " kW, Maintenance Recommended."), Low efficiency alert triggered, maintenance recommended.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.Timer;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.text.DecimalFormat;
import java.util.*;
import java.util.List;
```

```

import java.util.concurrent.ThreadLocalRandom;
/**
 * c120_WindTurbineSystemEx
 */
public class c120_WindTurbineSystemEx {
    // ===== Harness Toggles =====
    private static final boolean RUN_VT      = false;
    private static final boolean RUN_FT      = false;
    private static final boolean RUN_Z3      = false;
    private static final boolean RUN_UVT     = true;
    private static final boolean RUN_RANDOM  = false;
    private static final boolean RUN_GUI     = false;
    private static final boolean VERBOSE_CONSOLE = true;
    public static void main(String[] args) {
        TurbinePolicy policy = new TurbinePolicy.Builder()
            .setMaxPower(5000.0)
            .setMinPower(1000.0)
            .setOverflowMargin(250.0)
            .setMaxStepUp(2600.0)
            .setMaxStepDown(-2600.0)
            .setColdStartProtectPower(1200.0)
            .setHotDerateStartTemp(45.0)
            .setHotDerateSlope(0.02)
            .setRpmRampLimit(900.0)
            .setDivergenceWindow(12)
            .setOscillationWindow(6)
            .setLeakSlopeThreshold(-35.0)
            .setLowEfficiencyHeadroom(120.0)
            .setLogCapacity(20_000)
            .build();
        SensorSuite sensors = new SensorSuite(
            new WindSpeedSensor(6.5, 1.8, 20.0),
            new ShearSensor(0.12, 0.06),
            new AmbientTempSensor(20.0, 10.0),
            new AirDensityEstimator()
        );
        WindTurbineEx turbine = new WindTurbineEx(policy);
        // turbine.updateWithSensors(sensors.sample());
        // turbine.updateWithSensors(sensors.sample());
        // turbine.updateWithSensors(sensors.sample());
        // if (RUN_VT) TestSuites120.runVT(turbine);
        // if (RUN_FT) TestSuites120.runFT(turbine);
        // if (RUN_Z3) TestSuites120.runZ3(turbine);
        // if (RUN_UVT) TestSuites120.runUVT(turbine);
    }
}

```

```

        TestSuites120.runVT(turbine);
//        TestSuites120.runFT(turbine);
//        TestSuites120.runZ3(turbine);
//        TestSuites120.runUVT(turbine);
//        if (RUN_RANDOM) {
//            RandomSimulator sim = new RandomSimulator(sensors);
//            sim.runSteps(turbine, 200);
//        }
        turbine.printPowerLog();
        PowerStats stats = turbine.computeStats();
        if (VERBOSE_CONSOLE) {
            System.out.println("\n=== Summary Statistics ===");
            System.out.printf(Locale.US, "count=%d, powerMean=%.2f kW,
min=%.2f, max=%.2f, p95=%.2f\n",
                stats.count, stats.meanPower, stats.minPower, stats
.maxPower, stats.p95Power);
            System.out.printf(Locale.US, "deltaMean=%.2f, deltaMin=%.2f
, deltaMax=%.2f, deltaP95=%.2f\n",
                stats.meanDelta, stats.minDelta, stats.maxDelta, st
ats.p95Delta);
            System.out.printf(Locale.US, "var=%.2f, skew=%.3f, kurt=%.3
f\n",
                stats.variance, stats.skewness, stats.kurtosis);
        }
        try {
            String csvPath = "turbine_log_c120.csv";
            turbine.exportCsv(csvPath);
            if (VERBOSE_CONSOLE) System.out.println("CSV exported: " +
csvPath);
        } catch (IOException e) {
            System.out.println("CSV export failed: " + e.getMessage());
        }
        if (RUN_GUI) {
            SwingUtilities.invokeLater(() -> {
                TurbineFrame frame = new TurbineFrame(turbine, sensors)
;
                frame.setVisible(true);
            });
        }
    }
}

class PowerRecord {
    final long stepIndex;

```

```

    final double beforePower;
    final double delta;
    final double afterPower;
    final String message;
    final List<String> warnings;
    final SensorSnapshot snapshot;
    PowerRecord(long stepIndex,
                double beforePower,
                double delta,
                double afterPower,
                String message,
                List<String> warnings,
                SensorSnapshot snapshot) {
        this.stepIndex = stepIndex;
        this.beforePower = beforePower;
        this.delta = delta;
        this.afterPower = afterPower;
        this.message = message;
        this.warnings = warnings;
        this.snapshot = snapshot;
    }
}

class PowerStats {
    final int count;
    final double meanPower, minPower, maxPower, p95Power;
    final double meanDelta, minDelta, maxDelta, p95Delta;
    final double variance, skewness, kurtosis;
    PowerStats(int count,
                double meanPower, double minPower, double maxPower, double p95Power,
                double meanDelta, double minDelta, double maxDelta, double p95Delta,
                double variance, double skewness, double kurtosis) {
        this.count = count;
        this.meanPower = meanPower;
        this.minPower = minPower;
        this.maxPower = maxPower;
        this.p95Power = p95Power;
        this.meanDelta = meanDelta;
        this.minDelta = minDelta;
        this.maxDelta = maxDelta;
        this.p95Delta = p95Delta;
        this.variance = variance;
        this.skewness = skewness;
    }
}

```

```

        this.kurtosis = kurtosis;
    }
    static PowerStats from(List<PowerRecord> list) {
        if (list.isEmpty()) {
            return new PowerStats(0, Double.NaN, Double.NaN, Double.NaN,
, Double.NaN,
                                Double.NaN, Double.NaN, Double.NaN, Double.NaN,
                                Double.NaN, Double.NaN, Double.NaN);
        }
        List<Double> powers = new ArrayList<>(list.size());
        List<Double> deltas = new ArrayList<>(list.size());
        double minP = Double.POSITIVE_INFINITY, maxP = Double.NEGATIVE_
INFINITY;
        double minD = Double.POSITIVE_INFINITY, maxD = Double.NEGATIVE_
INFINITY;
        double sumP = 0, sumD = 0;
        for (PowerRecord r : list) {
            double p = r.afterPower;
            double d = r.delta;
            powers.add(p);
            deltas.add(d);
            sumP += p;
            sumD += d;
            minP = Math.min(minP, p);
            maxP = Math.max(maxP, p);
            minD = Math.min(minD, d);
            maxD = Math.max(maxD, d);
        }
        double meanP = sumP / powers.size();
        double meanD = sumD / deltas.size();
        Collections.sort(powers);
        Collections.sort(deltas);
        double p95P = powers.get((int)Math.floor(0.95 * (powers.size()-
1)));
        double p95D = deltas.get((int)Math.floor(0.95 * (deltas.size()-
1)));
        // 方差/偏度/峰度的简易实现（无偏/近似）
        double m2 = 0, m3 = 0, m4 = 0;
        for (double v : powers) {
            double x = v - meanP;
            double x2 = x * x;
            m2 += x2;
            m3 += x2 * x;
            m4 += x2 * x2;

```

```

    }
    int n = powers.size();
    double variance = n > 1 ? m2 / (n - 1) : 0.0;
    double skewness = (n > 2 && variance > 0)
        ? (Math.sqrt(n * (n - 1)) / (n - 2)) * (m3 / Math.pow(m
2, 1.5))
        : 0.0;
    double kurtosis = (n > 3 && variance > 0)
        ? (n * (n + 1) * m4 / (m2 * m2 * (n - 1) * (n - 2) * (n
- 3))) - 3.0 * ((n - 1)*(n - 1)) / ((n - 2)*(n - 3))
        : 0.0;
    return new PowerStats(n, meanP, minP, maxP, p95P, meanD, minD,
maxD, p95D, variance, skewness, kurtosis);
    }
}

class TurbinePolicy {
    final double maxPower;
    final double minPower;
    final double overflowMargin;
    final double maxStepUp;
    final double maxStepDown;
    final double coldStartProtectPower;
    final double hotDerateStartTemp;
    final double hotDerateSlope;
    final double rpmRampLimit;
    final int divergenceWindow;
    final int oscillationWindow;
    final double leakSlopeThreshold;
    final double lowEfficiencyHeadroom;
    final int logCapacity;
    private TurbinePolicy(Builder b) {
        this.maxPower = b.maxPower;
        this.minPower = b.minPower;
        this.overflowMargin = b.overflowMargin;
        this.maxStepUp = b.maxStepUp;
        this.maxStepDown = b.maxStepDown;
        this.coldStartProtectPower = b.coldStartProtectPower;
        this.hotDerateStartTemp = b.hotDerateStartTemp;
        this.hotDerateSlope = b.hotDerateSlope;
        this.rpmRampLimit = b.rpmRampLimit;
        this.divergenceWindow = b.divergenceWindow;
        this.oscillationWindow = b.oscillationWindow;
        this.leakSlopeThreshold = b.leakSlopeThreshold;
        this.lowEfficiencyHeadroom = b.lowEfficiencyHeadroom;
    }
}

```

```

        this.logCapacity = b.logCapacity;
    }
    boolean nearOverflow(double p) { return p > (maxPower - overflowMargin); }
    static class Builder {
        private double maxPower = 5000.0;
        private double minPower = 1000.0;
        private double overflowMargin = 250.0;
        private double maxStepUp = 2600.0;
        private double maxStepDown = -2600.0;
        private double coldStartProtectPower = 1200.0;
        private double hotDerateStartTemp = 45.0;
        private double hotDerateSlope = 0.02;
        private double rpmRampLimit = 900.0;
        private int divergenceWindow = 12;
        private int oscillationWindow = 6;
        private double leakSlopeThreshold = -35.0;
        private double lowEfficiencyHeadroom = 120.0;
        private int logCapacity = 20_000;
        Builder setMaxPower(double v){this.maxPower=v;return this;}
        Builder setMinPower(double v){this.minPower=v;return this;}
        Builder setOverflowMargin(double v){this.overflowMargin=v;return this;}
        Builder setMaxStepUp(double v){this.maxStepUp=v;return this;}
        Builder setMaxStepDown(double v){this.maxStepDown=v;return this;}
        Builder setColdStartProtectPower(double v){this.coldStartProtectPower=v;return this;}
        Builder setHotDerateStartTemp(double v){this.hotDerateStartTemp=v;return this;}
        Builder setHotDerateSlope(double v){this.hotDerateSlope=v;return this;}
        Builder setRpmRampLimit(double v){this.rpmRampLimit=v;return this;}
        Builder setDivergenceWindow(int v){this.divergenceWindow=v;return this;}
        Builder setOscillationWindow(int v){this.oscillationWindow=v;return this;}
        Builder setLeakSlopeThreshold(double v){this.leakSlopeThreshold=v;return this;}
        Builder setLowEfficiencyHeadroom(double v){this.lowEfficiencyHeadroom=v;return this;}
        Builder setLogCapacity(int v){this.logCapacity=v;return this;}
        TurbinePolicy build(){return new TurbinePolicy(this);}
    }

```

```

    }
}
class SensorSnapshot {
    final double windSpeed;    // m/s
    final double shearAlpha;   // -
    final double ambientTemp; // °C
    final double airDensity;   // kg/m^3
    final double idealPower;   // kW
    SensorSnapshot(double windSpeed, double shearAlpha, double ambientTemp, double airDensity, double idealPower) {
        this.windSpeed = windSpeed;
        this.shearAlpha = shearAlpha;
        this.ambientTemp = ambientTemp;
        this.airDensity = airDensity;
        this.idealPower = idealPower;
    }
}
class WindSpeedSensor {
    private final double mean;
    private final double sigma;
    private final double max;
    private final Random rnd = new Random(1207);
    WindSpeedSensor(double mean, double sigma, double max) {
        this.mean = mean; this.sigma = sigma; this.max = max;
    }
    double read() {
        double g = rnd.nextGaussian() * sigma + mean;
        if (g < 0) g = Math.abs(g) * 0.2;
        return Math.min(g, max);
    }
}
class ShearSensor {
    private final double meanAlpha;
    private final double sigma;
    private final Random rnd = new Random(1208);
    ShearSensor(double meanAlpha, double sigma){
        this.meanAlpha=meanAlpha;this.sigma=sigma;
    }
    double read() {
        double a = rnd.nextGaussian() * sigma + meanAlpha;
        if (a < 0) a = 0.02 + Math.abs(a)*0.2;
        return Math.min(a, 0.5);
    }
}

```

```

class AmbientTempSensor {
    private final double base;
    private final double amplitude;
    private int t = 0;
    AmbientTempSensor(double base, double amplitude){
        this.base=base;this.amplitude=amplitude;
    }
    double read() {
        t++;
        double phase = Math.sin((t % 1440) * Math.PI / 720.0);
        return base + amplitude * phase + ThreadLocalRandom.current().nextGaussian()*0.6;
    }
}

class AirDensityEstimator {
    double estimate(double tempC) {
        double rho = 1.293 - 0.00426 * tempC;
        if (rho < 0.9) rho = 0.9;
        if (rho > 1.35) rho = 1.35;
        return rho;
    }
}

class SensorSuite {
    private final WindSpeedSensor wind;
    private final ShearSensor shear;
    private final AmbientTempSensor temp;
    private final AirDensityEstimator rho;
    SensorSuite(WindSpeedSensor wind, ShearSensor shear, AmbientTempSensor temp, AirDensityEstimator rho) {
        this.wind = wind; this.shear = shear; this.temp = temp; this.rho = rho;
    }
    SensorSnapshot sample() {
        double v = wind.read();
        double a = shear.read();
        double t = this.temp.read();
        double density = rho.estimate(t);
        double k = 0.6;
        double idealKW = 0.5 * density * Math.pow(v, 3) * k;
        idealKW *= (1.0 + 0.4 * a);
        return new SensorSnapshot(v, a, t, density, idealKW);
    }
}

class PowerAnomalyDetector {

```

```

    static List<String> detect(TurbinePolicy p, double before, double d
elta, double after,
                                List<PowerRecord> hist, SensorSnapshot s
, boolean coldStart) {
        List<String> warns = new ArrayList<>();
        if (delta > p.maxStepUp) warns.add("Surge: step increase > maxS
tepUp");
        if (delta < p.maxStepDown) warns.add("Drop: step decrease < max
StepDown");
        if (after > p.maxPower) {
            warns.add("WARNING: Overload Detected! Emergency Shutdown R
ecommended.");
        } else if (after < p.minPower) {
            warns.add("ALERT: Low Efficiency! Maintenance Recommended."
);
        } else {
            if (p.nearOverflow(after)) warns.add("Caution: Near overloa
d margin");
            if (after < (p.minPower + p.lowEfficiencyHeadroom)) warns.a
dd("Advice: Low-efficiency headroom");
        }
        if (coldStart && after > p.coldStartProtectPower + 60) {
            warns.add("Cold start protection active");
        }
        if (s != null && s.ambientTemp >= p.hotDerateStartTemp) {
            warns.add("Thermal derate active");
        }
        if (hist.size() >= p.oscillationWindow) {
            if (isOscillating(hist, p.oscillationWindow)) {
                warns.add("Oscillation: alternating large deltas");
            }
        }
        if (hist.size() >= p.divergenceWindow) {
            double slope = estimateSlope(hist, p.divergenceWindow);
            if (slope < p.leakSlopeThreshold) {
                warns.add("Leak tendency: negative power trend");
            }
            if (slope > Math.abs(p.leakSlopeThreshold) * 0.9) {
                warns.add("Divergence: positive trend");
            }
        }
        if (Math.abs(delta) > p.rpmRampLimit) {
            warns.add("RPM ramp-coupled limiter suggested");
        }
    }

```

```

        return warns;
    }
    private static boolean isOscillating(List<PowerRecord> hist, int window) {
        int n = hist.size();
        int cnt = 0;
        for (int i = n - window + 1; i < n; i++) {
            double d1 = hist.get(i-1).delta;
            double d2 = hist.get(i).delta;
            if (Math.signum(d1) != Math.signum(d2) && Math.abs(d1) > 80
0 && Math.abs(d2) > 800) {
                cnt++;
            }
        }
        return cnt >= (window / 2);
    }
    private static double estimateSlope(List<PowerRecord> hist, int k)
{
    int n = Math.min(k, hist.size());
    double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;
    for (int i = 0; i < n; i++) {
        double x = i;
        double y = hist.get(hist.size() - 1 - i).afterPower;
        sumX += x; sumY += y; sumXY += x * y; sumX2 += x * x;
    }
    double denom = n * sumX2 - sumX * sumX;
    if (denom == 0) return 0;
    return (n * sumXY - sumX * sumY) / denom;
}
}
class WindTurbineEx {
    private final TurbinePolicy policy;
    private double currentPower = 3000.0;
    private final CircularBuffer<PowerRecord> log;
    private long step = 0;
    private boolean coldStart = true;
    WindTurbineEx(TurbinePolicy policy) {
        this.policy = policy;
        this.log = new CircularBuffer<>(policy.logCapacity);
    }
    public void updatePower(double changeAmount) {
        updateInternal(changeAmount, null);
    }
    public void updateWithSensors(SensorSnapshot s) {

```

```

        double target = applyThermalDerate(s.idealPower, s.ambientTemp)
;
        double delta = (target - currentPower) * 0.35;
        delta += ThreadLocalRandom.current().nextGaussian() * 120.0;
        updateInternal(delta, s);
    }
    private double applyThermalDerate(double ideal, double tempC) {
        if (tempC < policy.hotDerateStartTemp) return ideal;
        double over = tempC - policy.hotDerateStartTemp;
        double factor = Math.max(0.0, 1.0 - policy.hotDerateSlope * ove
r);
        return ideal * factor;
    }
    private void updateInternal(double changeAmount, SensorSnapshot sna
pshot) {
        double before = currentPower;
        double after = before + changeAmount;
        if (coldStart) {
            after = Math.min(after, policy.coldStartProtectPower);
        }
        String base = String.format(Locale.US,
            "Step #%d | Before: %.2f kW | Δ: %+,.2f kW | Current Po
wer Output: %.2f kW",
            ++step, before, changeAmount, after);
        List<PowerRecord> history = log.snapshotView();
        List<String> warns = PowerAnomalyDetector.detect(policy, before
, changeAmount, after, history, snapshot, coldStart);
        if (after > policy.maxPower) {
            after = policy.maxPower;
        } else if (after < policy.minPower) {
            after = policy.minPower;
        }
        StringBuilder sb = new StringBuilder(base);
        if (!warns.isEmpty()) {
            sb.append(" | ");
            for (int i = 0; i < warns.size(); i++) {
                if (i > 0) sb.append("; ");
                sb.append(warns.get(i));
            }
        }
        if (coldStart && history.size() >= 10) {
            boolean stable = true;
            for (int i=history.size()-10;i<history.size();i++){
                if (i<0) continue;

```

```

        if (Math.abs(history.get(i).delta) > 600) {stable=false
;break;}
    }
    if (stable) coldStart = false;
}
String line = sb.toString();
System.out.println(line);
currentPower = after;
PowerRecord record = new PowerRecord(step, before, changeAmount
, after, line, warns,
    snapshot != null ? snapshot : new SensorSnapshot(Double
.NaN, Double.NaN, Double.NaN, Double.NaN, Double.NaN));
    log.add(record);
}
public void printPowerLog() {
    System.out.println("\nPower Output Log:");
    for (PowerRecord r : log.iterable()) {
        System.out.println(r.message);
    }
}
public PowerStats computeStats() {
    return PowerStats.from(log.asList());
}
public void exportCsv(String file) throws IOException {
    try (PrintWriter pw = new PrintWriter(new BufferedWriter(new Fi
leWriter(file)))) {
        pw.println("step,before,delta,after,has_warning,warnings,wi
ndSpeed,shearAlpha,ambientTemp,airDensity,idealPower");
        for (PowerRecord r : log.iterable()) {
            String warn = String.join("|", r.warnings);
            pw.printf(Locale.US, "%d,%.6f,%.6f,%.6f,%b,%s,%.6f,%.6f
,%.6f,%.6f,%.6f%n",
                r.stepIndex, r.beforePower, r.delta, r.afterPow
er, !r.warnings.isEmpty(),
                escapeCsv(warn),
                r.snapshot.windSpeed, r.snapshot.shearAlpha, r.
snapshot.ambientTemp,
                r.snapshot.airDensity, r.snapshot.idealPower);
        }
    }
}
private static String escapeCsv(String s) {
    if (s == null) return "";
    if (s.contains(",") || s.contains("\\")) {

```

```

        return "\"" + s.replace("\\", "\\") + "\"";
    }
    return s;
}
}

class CircularBuffer<T> {
    private final Object[] arr;
    private int head = 0;
    private int size = 0;
    CircularBuffer(int capacity) {
        this.arr = new Object[Math.max(256, capacity)];
    }
    void add(T v) {
        arr[head] = v;
        head = (head + 1) % arr.length;
        if (size < arr.length) size++;
    }
    @SuppressWarnings("unchecked")
    List<T> asList() {
        List<T> list = new ArrayList<>(size);
        for (int i = size - 1; i >= 0; i--) {
            int idx = (head - 1 - i + arr.length) % arr.length;
            list.add((T) arr[idx]);
        }
        return list;
    }
    Iterable<T> iterable() {
        return this::iterator;
    }
    private Iterator<T> iterator() {
        List<T> snap = asList();
        return snap.iterator();
    }
    List<T> snapshotView() {
        return asList();
    }
}

class RandomSimulator {
    private final SensorSuite sensors;
    RandomSimulator(SensorSuite s){this.sensors=s;}
    void runSteps(WindTurbineEx t, int steps){
        for (int i=0;i<steps;i++){
            t.updateWithSensors(sensors.sample());
        }
    }
}

```

```

    }
}
class TurbineFrame extends JFrame {
    private final TurbinePanel panel;
    private final WindTurbineEx turbine;
    private final SensorSuite sensors;
    private volatile boolean running = true;
    private final Timer timer;
    TurbineFrame(WindTurbineEx turbine, SensorSuite sensors) {
        super("Wind Turbine Monitor (c120)");
        this.turbine = turbine;
        this.sensors = sensors;
        this.panel = new TurbinePanel();
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setSize(820, 520);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        add(panel, BorderLayout.CENTER);
        JPanel controls = new JPanel(new FlowLayout(FlowLayout.LEFT));
        controls.setBorder(new EmptyBorder(8,8,8,8));
        JButton stepBtn = new JButton("Step");
        JButton runBtn = new JButton("Run");
        JButton stopBtn = new JButton("Stop");
        JButton exportBtn = new JButton("Export CSV");
        controls.add(stepBtn);
        controls.add(runBtn);
        controls.add(stopBtn);
        controls.add(exportBtn);
        add(controls, BorderLayout.SOUTH);
        stepBtn.addActionListener(e -> doStep());
        runBtn.addActionListener(e -> running = true);
        stopBtn.addActionListener(e -> running = false);
        exportBtn.addActionListener(e -> {
            try {
                turbine.exportCsv("turbine_gui_export.csv");
                JOptionPane.showMessageDialog(this, "Exported to turbine_gui_export.csv");
            } catch (IOException ex) {
                JOptionPane.showMessageDialog(this, "Export failed: " + ex.getMessage());
            }
        });
        timer = new Timer(120, e -> {
            if (running) doStep();

```

```

    });
    addWindowListener(new WindowAdapter() {
        @Override public void windowOpened(WindowEvent e) { timer.start(); }
        @Override public void windowClosed(WindowEvent e) { timer.stop(); }
    });
}
private void doStep() {
    turbine.updateWithSensors(sensors.sample());
    panel.repaint();
}
}

class TurbinePanel extends JPanel {
    private final Deque<Double> series = new ArrayDeque<>();
    private static final int CAP = 160;
    private final DecimalFormat df = new DecimalFormat("#0");
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(new Color(245, 247, 250));
        g.fillRect(0,0,getWidth(),getHeight());
        g.setColor(new Color(220, 225, 232));
        for (int x=40;x<getWidth();x+=40) g.drawLine(x,20,x,getHeight()-40);
        for (int y=20;y<getHeight()-40;y+=40) g.drawLine(40,y,getWidth()-20,y);
        double last = series.isEmpty()? 3000.0 : series.peekLast();
        double next = last + ThreadLocalRandom.current().nextGaussian()*120.0;
        if (series.size()>=CAP) series.pollFirst();
        series.addLast(next);
        int w = getWidth()-60, h = getHeight()-80;
        int ox = 40, oy = getHeight()-40;
        double min = Collections.min(series);
        double max = Collections.max(series);
        double span = Math.max(1e-6, max-min);
        g.setColor(new Color(80,115,230));
        int i=0; int prevX=-1, prevY=-1;
        for (double v : series) {
            int x = ox + (int)((i*1.0/(CAP-1))*w);
            int y = oy - (int)((v - min)/span)*h);
            if (prevX>=0) g.drawLine(prevX, prevY, x, y);
            prevX = x; prevY = y; i++;
        }
    }
}

```

```

    }
    g.setColor(new Color(50,50,60));
    g.drawString("kW (scaled)", 10, 14);
    g.drawString("min="+df.format(min)+" max="+df.format(max), 10,
getHeight()-10);
    }
}
/* =====
*
* TESTCASE
* =====
*/
class TestSuites120 {
    static void runVT(WindTurbineEx t) {
        for (double d : VT_SEQ) t.updatePower(d);
    }
    static void runZ3(WindTurbineEx t) {
        for (double d : Z3_SEQ) t.updatePower(d);
    }
    static void runUVT(WindTurbineEx t) {
        for (double d : UVT_SEQ) t.updatePower(d);
    }
    static void runFT(WindTurbineEx t) {
        for (double d : FT_SEQ) t.updatePower(d);
    }
    // VT
    static final double[] VT_SEQ = new double[]{
        1749.09,
        1871.39,
        848.54,
        -114.94,
        4715.32,
        3321.22,
        4162.25,
        2987.27,
        838.73,
        1294.71,
        3997.23,
        3182.79,
        2705.31,
        2236.73,
        1209.44,
        2352.24,
        4601.53,
        1121.82,

```

322.70,
1341.35,
2534.90,
2774.78,
4.14,
2806.00,
1915.71,
4259.55,
-165.65,
-486.31,
1290.56,
159.10,
4641.60,
1535.14,
3666.31,
677.27,
2823.32,
1687.12,
4405.64,
4412.69,
3662.97,
3138.70,
1323.36,
562.43,
-51.27,
581.94,
-480.82,
872.87,
1430.69,
227.82,
1351.88,
293.08,
2382.67,
2888.49,
480.60,
2093.71,
404.15,
3325.59,
39.41,
4699.33,
3514.16,
1575.54,
1707.54,
2977.12,

```
1105.64,  
2312.75,  
-273.48,  
1722.37,  
2448.37,  
165.68,  
2334.08,  
2549.31,  
-402.81,  
3241.35,  
2454.18,  
3074.26,  
666.35,  
2811.20,  
2906.31,  
2767.89,  
1937.13,  
2563.94,  
4366.95,  
4185.55,  
3743.21,  
4065.13,  
2691.75,  
4901.75,  
606.50,  
-470.73,  
4076.74,  
3029.74,  
3838.44,  
524.05,  
4632.46,  
3719.06,  
1251.38,  
2737.23,  
3411.40,  
279.47,  
4264.34,  
4906.42,  
  
};  
// Z3  
static final double[] Z3_SEQ = new double[]{  
    -47.70,  
    419.47,  
    340.35,
```

-394.61,
-84.99,
-305.24,
156.97,
-472.42,
-438.21,
437.73,
-354.21,
72.38,
147.96,
104.37,
-20.31,
-388.59,
-114.67,
174.98,
-160.65,
289.97,
269.40,
178.54,
-104.10,
200.84,
-229.45,
-17.38,
-70.57,
-345.08,
425.23,
-414.21,
-498.72,
-425.69,
-438.79,
455.23,
-255.52,
-227.24,
-252.20,
264.10,
33.22,
-223.43,
-25.20,
-441.32,
220.57,
376.25,
281.18,
122.68,
280.63,

218.11,
312.72,
398.77,
-181.28,
230.93,
-333.42,
-20.91,
-488.63,
-463.68,
-233.82,
-484.67,
-480.63,
426.35,
170.62,
256.56,
43.13,
-343.52,
118.66,
474.40,
-204.33,
314.81,
64.80,
67.03,
-181.06,
390.08,
18.97,
345.67,
-433.96,
-135.57,
-321.85,
-180.88,
-129.64,
-376.48,
-465.26,
-4.81,
492.35,
421.69,
-435.78,
411.95,
-473.94,
203.66,
-185.68,
-155.07,
307.43,

```
-328.31,  
90.71,  
331.06,  
95.49,  
63.98,  
-103.68,  
104.26,  
-160.39,  
-261.78  
};  
//UVT  
static final double[] UVT_SEQ = new double[]{  
-47.70,  
419.47,  
340.35,  
-394.61,  
-84.99,  
-305.24,  
156.97,  
-472.42,  
-438.21,  
437.73,  
-354.21,  
72.38,  
147.96,  
104.37,  
-20.31,  
-388.59,  
-114.67,  
174.98,  
-160.65,  
289.97,  
269.40,  
178.54,  
-104.10,  
200.84,  
-229.45,  
-17.38,  
-70.57,  
-345.08,  
425.23,  
-414.21,  
-498.72,  
-425.69,
```

-438.79,
455.23,
-255.52,
-227.24,
-252.20,
264.10,
33.22,
-223.43,
-25.20,
-441.32,
220.57,
376.25,
281.18,
122.68,
280.63,
218.11,
312.72,
398.77,
-181.28,
230.93,
-333.42,
-20.91,
-488.63,
-463.68,
-233.82,
-484.67,
-480.63,
426.35,
170.62,
256.56,
43.13,
-343.52,
118.66,
474.40,
-204.33,
314.81,
64.80,
67.03,
-181.06,
390.08,
18.97,
345.67,
-433.96,
-135.57,

[illegible]

[illegible][illegible][illegible][illegible]

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

[illegible]

0.00,

0.00,

0.00,

0.00,

0.00,

—

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

[illegible]

—

[illegible]

—

[illegible][illegible]

—

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

0.00,

0.00,

0.00,

0.00,

0.00,

[illegible]

0.00,

0.00,

0.00,

[illegible][illegible]

0.00,

[illegible][illegible][illegible]

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c120_WindTurbineSystemEx.java	78.9 %	3,116	834	3,950
> AirDensityEstimator	13.0 %	3	20	23
> SensorSuite	24.6 %	15	46	61
> AmbientTempSensor	28.6 %	12	30	42
> ShearSensor	37.5 %	15	25	40
> WindSpeedSensor	42.9 %	18	24	42
> PowerAnomalyDetector	83.4 %	236	47	283
> WindTurbineEx	83.4 %	367	73	440
> PowerStats	94.3 %	316	19	335
> c120_WindTurbineSystemEx	95.1 %	175	9	184
> TestSuites120	96.3 %	1,633	63	1,696
> TurbinePolicy	97.2 %	69	2	71
> CircularBuffer<T>	100.0 %	95	0	95
> PowerRecord	100.0 %	24	0	24
> SensorSnapshot	100.0 %	18	0	18

4. Fuzz testing branch coverage: 52.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c120_WindTurbineSystemEx.java	42.4 %	61	83	144
> AirDensityEstimator	0.0 %	0	4	4
> ShearSensor	0.0 %	0	2	2
> WindSpeedSensor	0.0 %	0	2	2
> TestSuites120	25.0 %	2	6	8
> TurbinePolicy	50.0 %	1	1	2
> WindTurbineEx	52.5 %	21	19	40
> PowerAnomalyDetector	73.9 %	34	12	46
> CircularBuffer<T>	75.0 %	3	1	4
> AmbientTempSensor		0	0	0
> c120_WindTurbineSystemEx		0	0	0
> PowerRecord		0	0	0
> SensorSnapshot		0	0	0
> SensorSuite		0	0	0

5. Fuzz testing instruction coverage: 52.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c120_WindTurbineSystemEx.java	66.8 %	2,560	1,275	3,835
> AirDensityEstimator	13.0 %	3	20	23
> SensorSuite	24.6 %	15	46	61
> AmbientTempSensor	28.6 %	12	30	42
> ShearSensor	37.5 %	15	25	40
> WindSpeedSensor	42.9 %	18	24	42
> WindTurbineEx	52.5 %	231	209	440
> PowerAnomalyDetector	85.2 %	241	42	283
> c120_WindTurbineSystemEx	95.7 %	66	3	69
> TestSuites120	96.3 %	1,633	63	1,696
> TurbinePolicy	97.2 %	69	2	71
> CircularBuffer<T>	100.0 %	95	0	95
> PowerRecord	100.0 %	24	0	24
> SensorSnapshot	100.0 %	18	0	18

6. Z3 branch coverage: 57.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c120_WindTurbineSystemEx.java	41.7 %	60	84	144
> AirDensityEstimator	0.0 %	0	4	4
> ShearSensor	0.0 %	0	2	2
> WindSpeedSensor	0.0 %	0	2	2
> TestSuites120	25.0 %	2	6	8
> TurbinePolicy	50.0 %	1	1	2
> WindTurbineEx	57.5 %	23	17	40
> PowerAnomalyDetector	67.4 %	31	15	46
> CircularBuffer<T>	75.0 %	3	1	4
> AmbientTempSensor		0	0	0
> c120_WindTurbineSystemEx		0	0	0
> PowerRecord		0	0	0
> SensorSnapshot		0	0	0
> SensorSuite		0	0	0

7. Z3 instruction coverage: 52.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c120_WindTurbineSystemEx.java	66.5 %	2,550	1,285	3,835
> AirDensityEstimator	13.0 %	3	20	23
> SensorSuite	24.6 %	15	46	61
> AmbientTempSensor	28.6 %	12	30	42
> ShearSensor	37.5 %	15	25	40
> WindSpeedSensor	42.9 %	18	24	42
> WindTurbineEx	52.5 %	231	209	440
> PowerAnomalyDetector	81.6 %	231	52	283
> c120_WindTurbineSystemEx	95.7 %	66	3	69
> TestSuites120	96.3 %	1,633	63	1,696
> TurbinePolicy	97.2 %	69	2	71
> CircularBuffer<T>	100.0 %	95	0	95
> PowerRecord	100.0 %	24	0	24
> SensorSnapshot	100.0 %	18	0	18

8. Unimproved V-method branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c120_WindTurbineSystemEx.java	53.5 %	77	67	144
> AirDensityEstimator	0.0 %	0	4	4
> ShearSensor	0.0 %	0	2	2
> WindSpeedSensor	0.0 %	0	2	2
> TestSuites120	25.0 %	2	6	8
> TurbinePolicy	50.0 %	1	1	2
> PowerStats	62.5 %	10	6	16
> PowerAnomalyDetector	67.4 %	31	15	46
> CircularBuffer<T>	75.0 %	3	1	4
> WindTurbineEx	75.0 %	30	10	40
> AmbientTempSensor		0	0	0
> c120_WindTurbineSystemEx		0	0	0
> PowerRecord		0	0	0
> SensorSnapshot		0	0	0
> SensorSuite		0	0	0

9. Unimproved V-method instruction coverage: 83.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c120_WindTurbineSystemEx.java	78.8 %	3,111	839	3,950
> AirDensityEstimator	13.0 %	3	20	23
> SensorSuite	24.6 %	15	46	61
> AmbientTempSensor	28.6 %	12	30	42
> ShearSensor	37.5 %	15	25	40
> WindSpeedSensor	42.9 %	18	24	42
> PowerAnomalyDetector	81.6 %	231	52	283
> WindTurbineEx	83.4 %	367	73	440
> PowerStats	94.3 %	316	19	335
> c120_WindTurbineSystemEx	95.1 %	175	9	184
> TestSuites120	96.3 %	1,633	63	1,696
> TurbinePolicy	97.2 %	69	2	71
> CircularBuffer<T>	100.0 %	95	0	95
> PowerRecord	100.0 %	24	0	24
> SensorSnapshot	100.0 %	18	0	18